

Overview

Interview Formats 🎧

The two main formats you'll encounter: platform-based (CoderPad, HackerRank) and BYOE (Bring Your Own Environment).

AI-enabled coding interviews come in two flavors, and knowing which one you're walking into matters a lot. The tools available to you, the types of problems you'll face, and even the strategies that work best are all different depending on the format.

1. **Structured** (CoderPad, HackerRank) where the company provides the environment, AI tools, and a defined problem structure
2. **Open-Ended** where you use your own IDE, your own AI tools, and build from scratch while sharing your screen

Ask your recruiter which one it is. If they don't know, push until you get an answer.

Structured

Try Structured AI-Coding

Practice real problems in an AI-enabled environment

 Open Sandbox

In a structured interview, the company provides the coding environment and the AI assistant. You open a browser, log into CoderPad or HackerRank, and everything is already there: the codebase, the AI chat panel, and the problem statement.

These interviews follow a defined problem arc. The typical format runs three phases: start by fixing a bug in the existing code, implement a new feature on top of the working codebase, then optimize for larger inputs or edge cases. Knowing that structure early lets you pace yourself. You know roughly what's coming and can avoid burning all your time on phase one.

Meta is the most well-known example, in part because they were among the first major companies to standardize it and their CoderPad setup has become the reference point candidates compare everything else to. Their interview uses CoderPad with a selection of AI

models (GPT-4o, GPT-5, Claude Sonnet/Haiku, Gemini, Llama). LinkedIn also uses CoderPad, with their own model selection and the same multi-phase structure. Uber runs on HackerRank with a pre-set repository, where you're expected to navigate existing code before making changes.

The downside is that you're locked into whatever tools they give you. If the AI assistant isn't great (and candidates at Meta and Uber have consistently reported it being worse than what they practiced with), you can't switch to something better. You work with what you've got.



Both Meta and LinkedIn offer practice sessions on their platform before the real interview. Always take advantage of these. The CoderPad AI interface has quirks you won't encounter in your normal IDE, and spending 30 minutes getting comfortable with it beforehand is time well spent.

Open-Ended

Try Open-Ended AI-Coding

Practice real problems in an AI-enabled environment

Open Sandbox

You use your own IDE, your own AI tools, and share your screen. This format is more common at companies that want to see how you actually work day-to-day. They're not interested in whether you can adapt to a constrained platform, they want to know what your real workflow looks like.

Shopify is the canonical example: they tell you to use whatever you'd normally use at work. The specifics vary across companies that use this format. Some mandate AI usage and score it explicitly (Canva). Some have you build features incrementally against test cases (Rippling). Some vary by team, from structured DSA to open repo work (Adobe).

The upside is that you're working with tools you already know. You don't have to learn an unfamiliar interface under pressure, and you can fall back on workflows you've built over time.

The downside is that the interviewer sees *everything*. Your prompts, your fumbling between tools, that moment where you accidentally paste something into the wrong window. There's no hiding behind a platform. Your workflow is completely visible, which means it needs to actually be good.

The problems here are greenfield. You might get an empty repository and a problem statement, or just a README describing what to build. You start from nothing and build a working application. The emphasis shifts from "can you navigate existing code" to "can you design and build something from scratch with AI as your accelerator."

This tends to require candidates to leverage many of the skills we cover in our [Low Level Design content](#), by showing that you can design software that is both functional and extensible, even if the majority of the actual implementation is handled by the AI.

What to ask your recruiter

Before your interview, get clear answers to these questions:

What's the format?

- Structured or Open-Ended? If the recruiter says "AI-enabled" but doesn't specify, ask directly.
- Is there starter code or are you building from scratch?

If it's structured

- Which platform (CoderPad, HackerRank)?
- Is there a practice environment you can access beforehand?
- How many phases does the interview have?

If it's open-ended

- Are there restrictions on which AI tools you can use?
- Does the company have a preference (some suggest Cursor specifically)?
- Will you need to push code to a repository, or is screen share enough?



Recruiters are usually happy to give you format details. They want you to succeed. If yours is vague, try asking "what should I have set up on my machine before the interview?" That

question tends to surface the specifics.